```
PROGRAM PL0(INPUT, OUTPUT);
↱PL/0 COMPILER WITH CODE GENERATION↓
LABEL 99;
CONST NORW = 11;    ↱NO. OF RESERVED WORDS↓
   TXMAX = 100;       ↱LENGTH OF IDENTIFIER TABLE↓
   NMAX = 14;          ↱MAX NO. OF DIGITS IN NUMBERS↓
   AL = 10;              ↱LENGTH OF IDENTIFIERS↓
   AMAX = 2047;       ↱MAXIMUM ADDRESS↓
   LEVMAX = 3;         ↱MAXIMUM DEPTH OF BLOCK NESTING↓
   CXMAX = 200;       ↱SIZE OF CODE ARRAY↓
TYPE SYMBOL =
   (NUL, IDENT, NUMBER, PLUS, MINUS, TIMES, SLASH, ODDSYM,
     EQL, NEQ, LSS, LEQ, GTR, GEQ, LPAREN, RPAREN, COMMA, SEMICOLON,
     PERIOD, BECOMES, BEGINSYM, ENDSYM, IFSYM, THENSYM,
     WHILESYM, DOSYM, CALLSYM, CONSTSYM, VARSYM, PROCSYM);
   ALFA = PACKED ARRAY [1..AL] OF CHAR;
   OBJECT = (CONSTANT, VARIABLE, PROZEDURE);
   SYMSET = SET OF SYMBOL;
   FCT = (LIT, OPR, LOD, STO, CAL, INT, JMP, JPC);    ↱FUNCTIONS↓
   INSTRUCTION = PACKED RECORD
                              F: FCT;              ↱FUNCTION CODE↓
                              L: 0..LEVMAX;    ↱LEVEL↓
                              A: 0..AMAX       ↱DISPLACEMENT ADDRESS↓
                            END;
↱ LIT 0, A    :    LOAD CONSTANT A
   OPR 0, A    :    EXECUTE OPERATION A
   LOD L, A    :    LOAD VARIABLE L, A
   STO L, A    :    STORE VARIABLE L, A
   CAL L, A    :    CALL PROCEDURE A AT LEVEL L
   INT 0, A    :    INCREMENT T-REGISTER BY A
   JMP 0, A    :    JUMP TO A
   JPC 0, A    :    JUMP CONDITIONAL TO A    ↓
VAR CH: CHAR;              ↱LAST CHARACTER READ↓
   SYM: SYMBOL;            ↱LAST SYMBOL READ↓
   ID: ALFA;                ↱LAST IDENTIFIER READ↓
   NUM: INTEGER;           ↱LAST NUMBER READ↓
   CC: INTEGER;            ↱CHARACTER COUNT↓
   LL: INTEGER;            ↱LINE LENGTH↓
   KK, ERR: INTEGER;
   CX: INTEGER;            ↱CODE ALLOCATION INDEX↓
   LINE: ARRAY [1..81] OF CHAR;
   A: ALFA;
   CODE: ARRAY [0..CXMAX] OF INSTRUCTION;
   WORD: ARRAY [1..NORW] OF ALFA;
   WSYM: ARRAY [1..NORW] OF SYMBOL;
   SSYM: ARRAY [CHAR] OF SYMBOL;
```

```
   MNEMONIC: ARRAY [FCT] OF
                     PACKED ARRAY [1..5] OF CHAR;
   DECLBEGSYS, STATBEGSYS, FACBEGSYS: SYMSET;
   TABLE: ARRAY [0..TXMAX] OF
             RECORD NAME: ALFA;
                CASE KIND: OBJECT OF
                   CONSTANT: (VAL: INTEGER);
                   VARIABLE, PROZEDURE: (LEVEL, ADR: INTEGER)
             END;
PROCEDURE ERROR(N: INTEGER);
BEGIN WRITELN(≡ ****≡, ≡  ≡: CC - 1, ≡↑≡, N: 2); ERR := ERR + 1
END ↱ERROR↳;
PROCEDURE GETSYM;
   VAR I, J, K: INTEGER;
   PROCEDURE GETCH;
   BEGIN IF CC = LL THEN
            BEGIN IF EOF(INPUT) THEN
                     BEGIN WRITE(≡ PROGRAM INCOMPLETE≡); GOTO 99
                     END;
               LL := 0; CC := 0; WRITE(CX: 5, ≡  ≡);
               WHILE ¬EOLN(INPUT) DO
               BEGIN LL := LL + 1; READ(CH); WRITE(CH); LINE[LL] := CH
               END;
               WRITELN; LL := LL + 1; READ(LINE[LL])
            END;
      CC := CC + 1; CH := LINE[CC]
   END ↱GETCH↳;
BEGIN ↱GETSYM↳
   WHILE CH = ≡  ≡ DO GETCH;
   IF CH IN [≡A≡..≡Z≡] THEN
   BEGIN ↱IDENTIFIER OR RESERVED WORD↳ K := 0;
      REPEAT IF K < AL THEN
                BEGIN K := K + 1; A[K] := CH
                END;
         GETCH
      UNTIL ¬(CH IN [≡A≡..≡Z≡, ≡0≡..≡9≡]);
      IF K ≥ KK THEN KK := K ELSE
         REPEAT A[KK] := ≡  ≡; KK := KK - 1
         UNTIL KK = K;
      ID := A; I := 1; J := NORW;
      REPEAT K := (I + J) DIV 2;
         IF ID ≤ WORD[K] THEN J := K - 1;
         IF ID ≥ WORD[K] THEN I := K + 1
      UNTIL I > J;
      IF I - 1 > J THEN SYM := WSYM[K] ELSE SYM := IDENT
   END ELSE
```

```
   IF CH IN [≡0≡..≡9≡] THEN
   BEGIN ↱NUMBER↳ K := 0; NUM := 0; SYM := NUMBER;
      REPEAT NUM := 10 * NUM + (ORD(CH) - ORD(≡0≡));
         K := K + 1; GETCH
      UNTIL ¬(CH IN [≡0≡..≡9≡]);
      IF K > NMAX THEN ERROR(30)
   END ELSE
   IF CH = ≡:≡ THEN
   BEGIN GETCH;
      IF CH = ≡=≡ THEN
      BEGIN SYM := BECOMES; GETCH
      END ELSE SYM := NUL
   END ELSE
   BEGIN SYM := SSYM[CH]; GETCH
   END
END ↱GETSYM↳;
PROCEDURE GEN(X: FCT; Y, Z: INTEGER);
BEGIN IF CX > CXMAX THEN
         BEGIN WRITE(≡ PROGRAM TOO LONG≡); GOTO 99
         END;
   WITH CODE[CX] DO
   BEGIN F := X; L := Y; A := Z
   END;
   CX := CX + 1
END ↱GEN↳;
PROCEDURE TEST(S1, S2: SYMSET; N: INTEGER);
BEGIN IF ¬(SYM IN S1) THEN
         BEGIN ERROR(N); S1 := S1 + S2;
            WHILE ¬(SYM IN S1) DO GETSYM
         END
END ↱TEST↳;
PROCEDURE BLOCK(LEV, TX: INTEGER; FSYS: SYMSET);
   VAR DX: INTEGER;    ↱DATA ALLOCATION INDEX↳
      TX0: INTEGER;     ↱INITIAL TABLE INDEX↳
      CX0: INTEGER;     ↱INITIAL CODE INDEX↳
   PROCEDURE ENTER(K: OBJECT);
   BEGIN ↱ENTER OBJECT INTO TABLE↳
      TX := TX + 1;
      WITH TABLE[TX] DO
      BEGIN NAME := ID; KIND := K;
         CASE K OF
            CONSTANT: BEGIN IF NUM > AMAX THEN
                              BEGIN ERROR(30); NUM := 0 END;
                           VAL := NUM
                        END;
            VARIABLE: BEGIN LEVEL := LEV; ADR := DX; DX := DX + 1;
```

```
                        END;
          PROZEDURE: LEVEL := LEV
       END
    END
END ↱ENTER↳;
FUNCTION POSITION(ID: ALFA): INTEGER;
   VAR I: INTEGER;
BEGIN ↱FIND IDENTIFIER ID IN TABLE↳
   TABLE[0].NAME := ID; I := TX;
   WHILE TABLE[I].NAME ≠ ID DO I := I - 1;
   POSITION := I
END ↱POSITION↳;
PROCEDURE CONSTDECLARATION;
BEGIN IF SYM = IDENT THEN
          BEGIN GETSYM;
             IF SYM IN [EQL, BECOMES] THEN
             BEGIN IF SYM = BECOMES THEN ERROR(1);
                GETSYM;
                IF SYM = NUMBER THEN
                BEGIN ENTER(CONSTANT); GETSYM
                END ELSE ERROR(2)
             END ELSE ERROR(3)
          END ELSE ERROR(4)
END ↱CONSTDECLARATION↳;
PROCEDURE VARDECLARATION;
BEGIN IF SYM = IDENT THEN
          BEGIN ENTER(VARIABLE); GETSYM
          END ELSE ERROR(4)
END ↱VARDECLARATION↳;
PROCEDURE LISTCODE;
   VAR I: INTEGER;
BEGIN ↱LIST CODE GENERATED FOR THIS BLOCK↳
   FOR I := CX0 TO CX - 1 DO
       WITH CODE[I] DO
       WRITELN(I, MNEMONIC[F]: 5, L: 3, A: 5)
END ↱LISTCODE↳;
PROCEDURE STATEMENT(FSYS: SYMSET);
   VAR I, CX1, CX2: INTEGER;
   PROCEDURE EXPRESSION(FSYS: SYMSET);
       VAR ADDOP: SYMBOL;
       PROCEDURE TERM(FSYS: SYMSET);
          VAR MULOP: SYMBOL;
          PROCEDURE FACTOR (FSYS: SYMSET);
             VAR I: INTEGER;
          BEGIN TEST(FACBEGSYS, FSYS, 24);
             WHILE SYM IN FACBEGSYS DO
```

```
        BEGIN
          IF SYM = IDENT THEN
          BEGIN I := POSITION(ID);
            IF I = 0 THEN ERROR(11) ELSE
            WITH TABLE[I] DO
            CASE KIND OF
              CONSTANT: GEN(LIT, 0, VAL);
              VARIABLE: GEN(LOD, LEV - LEVEL, ADR);
              PROZEDURE: ERROR(21)
            END;
            GETSYM
          END ELSE
          IF SYM = NUMBER THEN
          BEGIN IF NUM > AMAX THEN
                BEGIN ERROR(30); NUM := 0
                END;
            GEN(LIT, 0, NUM); GETSYM
          END ELSE
          IF SYM = LPAREN THEN
          BEGIN GETSYM; EXPRESSION([RPAREN] + FSYS);
            IF SYM = RPAREN THEN GETSYM ELSE ERROR(22)
          END;
          TEST(FSYS, [LPAREN], 23)
        END
      END ↱FACTOR↳;
    BEGIN ↱TERM↳ FACTOR(FSYS + [TIMES, SLASH]);
      WHILE SYM IN [TIMES, SLASH] DO
      BEGIN MULOP := SYM; GETSYM; FACTOR(FSYS + [TIMES, SLASH]);
        IF MULOP = TIMES THEN GEN(OPR, 0, 4) ELSE GEN(OPR, 0, 5)
      END
    END ↱TERM↳;
  BEGIN ↱EXPRESSION↳
    IF SYM IN [PLUS, MINUS] THEN
    BEGIN ADDOP := SYM; GETSYM; TERM(FSYS + [PLUS, MINUS]);
      IF ADDOP = MINUS THEN GEN(OPR, 0, 1)
    END ELSE TERM(FSYS + [PLUS, MINUS]);
    WHILE SYM IN [PLUS, MINUS] DO
    BEGIN ADDOP := SYM; GETSYM; TERM(FSYS + [PLUS, MINUS]);
      IF ADDOP = PLUS THEN GEN(OPR, 0, 2) ELSE GEN(OPR, 0, 3)
    END
  END ↱EXPRESSION↳;
  PROCEDURE CONDITION(FSYS: SYMSET);
    VAR RELOP: SYMBOL;
  BEGIN
    IF SYM = ODDSYM THEN
    BEGIN GETSYM; EXPRESSION(FSYS); GEN(OPR, 0, 6)
```

```
      END ELSE
      BEGIN EXPRESSION([EQL, NEQ, LSS, GTR, LEQ, GEQ] + FSYS);
        IF ¬(SYM IN [EQL, NEQ, LSS, LEQ, GTR, GEQ]) THEN
          ERROR(20) ELSE
        BEGIN RELOP := SYM; GETSYM; EXPRESSION(FSYS);
          CASE RELOP OF
            EQL: GEN(OPR, 0, 8);
            NEQ: GEN(OPR, 0, 9);
            LSS: GEN(OPR, 0, 10);
            GEQ: GEN(OPR, 0, 11);
            GTR: GEN(OPR, 0, 12);
            LEQ: GEN(OPR, 0, 13)
          END
        END
      END
    END ↱CONDITION↳;
BEGIN ↱STATEMENT↳
    IF SYM = IDENT THEN
    BEGIN I := POSITION(ID);
      IF I = 0 THEN ERROR(11) ELSE
      IF TABLE[I].KIND ≠ VARIABLE THEN
        BEGIN ↱ASSIGNMENT TO NON-VARIABLE↳ ERROR(12); I := 0
        END;
      GETSYM; IF SYM = BECOMES THEN GETSYM ELSE ERROR(13);
      EXPRESSION(FSYS);
      IF I ≠ 0 THEN
        WITH TABLE[I] DO GEN(STO, LEV - LEVEL, ADR)
    END ELSE
    IF SYM = CALLSYM THEN
    BEGIN GETSYM;
      IF SYM ≠ IDENT THEN ERROR(14) ELSE
      BEGIN I := POSITION(ID);
        IF I = 0 THEN ERROR(11) ELSE
        WITH TABLE[I] DO
        IF KIND = PROZEDURE THEN GEN(CAL, LEV - LEVEL, ADR)
        ELSE ERROR(15);
        GETSYM
      END
    END ELSE
    IF SYM = IFSYM THEN
    BEGIN GETSYM; CONDITION([THENSYM, DOSYM] + FSYS);
      IF SYM = THENSYM THEN GETSYM ELSE ERROR(16);
      CX1 := CX; GEN(JPC, 0, 0);
      STATEMENT(FSYS); CODE[CX1].A := CX
    END ELSE
    IF SYM = BEGINSYM THEN
```

```
      BEGIN GETSYM; STATEMENT([SEMICOLON, ENDSYM] + FSYS);
        WHILE SYM IN [SEMICOLON] + STATBEGSYS DO
        BEGIN
          IF SYM = SEMICOLON THEN GETSYM ELSE ERROR(10);
          STATEMENT([SEMICOLON, ENDSYM] + FSYS)
        END;
        IF SYM = ENDSYM THEN GETSYM ELSE ERROR(17)
      END ELSE
      IF SYM = WHILESYM THEN
      BEGIN CX1 := CX; GETSYM; CONDITION([DOSYM] + FSYS);
        CX2 := CX; GEN(JPC, 0, 0);
        IF SYM = DOSYM THEN GETSYM ELSE ERROR(18);
        STATEMENT(FSYS); GEN(JMP, 0, CX1); CODE[CX2].A := CX
      END;
      TEST(FSYS, [], 19)
    END ↱STATEMENT↳;
  BEGIN ↱BLOCK↳ DX := 3; TX0 := TX; TABLE[TX].ADR := CX; GEN(JMP, 0, 0);
    IF LEV > LEVMAX THEN ERROR(32);
    REPEAT
      IF SYM = CONSTSYM THEN
      BEGIN GETSYM;
        REPEAT CONSTDECLARATION;
          WHILE SYM = COMMA DO
          BEGIN GETSYM; CONSTDECLARATION
          END;
          IF SYM = SEMICOLON THEN GETSYM ELSE ERROR(5)
        UNTIL SYM ≠ IDENT
      END;
      IF SYM = VARSYM THEN
      BEGIN GETSYM;
        REPEAT VARDECLARATION;
          WHILE SYM = COMMA DO
          BEGIN GETSYM; VARDECLARATION
          END;
          IF SYM = SEMICOLON THEN GETSYM ELSE ERROR(5)
        UNTIL SYM ≠ IDENT
      END;
      WHILE SYM = PROCSYM DO
      BEGIN GETSYM;
        IF SYM = IDENT THEN
        BEGIN ENTER(PROZEDURE); GETSYM
        END ELSE ERROR(4);
        IF SYM = SEMICOLON THEN GETSYM ELSE ERROR(5);
        BLOCK(LEV + 1, TX, [SEMICOLON] + FSYS);
        IF SYM = SEMICOLON THEN
        BEGIN GETSYM; TEST(STATBEGSYS + [IDENT, PROCSYM], FSYS, 6)
```

```
        END ELSE ERROR(5)
      END;
      TEST(STATBEGSYS + [IDENT], DECLBEGSYS, 7)
   UNTIL ¬(SYM IN DECLBEGSYS);
   CODE[TABLE[TX0].ADR].A := CX;
   WITH TABLE[TX0] DO
   ADR := CX; ↱START ADR OF CODE↳
   CX0 := CX; GEN(INT, 0, DX);
   STATEMENT([SEMICOLON, ENDSYM] + FSYS);
   GEN(OPR, 0, 0); ↱RETURN↳
   TEST(FSYS, [], 8);
   LISTCODE
END ↱BLOCK↳;
PROCEDURE INTERPRET;
   CONST STACKSIZE = 500;
   VAR P, B, T: INTEGER;    ↱PROGRAM-, BASE-, TOPSTACK- REGISTERS↳
      I: INSTRUCTION; ↱INSTRUCTION REGISTER↳
      S: ARRAY [1..STACKSIZE] OF INTEGER;    ↱DATASTORE↳
   FUNCTION BASE(L: INTEGER): INTEGER;
      VAR B1: INTEGER;
   BEGIN B1 := B; ↱FIND BASE L LEVELS DOWN↳
      WHILE L > 0 DO
      BEGIN B1 := S[B1]; L := L - 1
      END;
      BASE := B1
   END ↱BASE↳;
BEGIN WRITELN(≡ START PL/0≡);
   T := 0; B := 1; P := 0;
   S[1] := 0; S[2] := 0; S[3] := 0;
   REPEAT I := CODE[P]; P := P + 1;
      WITH I DO
      CASE F OF
         LIT: BEGIN T := T + 1; S[T] := A
                 END;
         OPR: CASE A OF    ↱OPERATOR↳
                    0: BEGIN    ↱RETURN↳
                          T := B - 1; P := S[T + 3]; B := S[T + 2];
                       END;
                    1: S[T] := -S[T];
                    2: BEGIN T := T - 1; S[T] := S[T] + S[T + 1]
                       END;
                    3: BEGIN T := T - 1; S[T] := S[T] - S[T + 1]
                       END;
                    4: BEGIN T := T - 1; S[T] := S[T] * S[T + 1]
                       END;
                    5: BEGIN T := T - 1; S[T] := S[T] DIV S[T + 1]
```

```
                END;
            6: S[T] := ORD(ODD(S[T]));
            8: BEGIN T := T - 1; S[T] := ORD(S[T] = S[T + 1])
                END;
            9: BEGIN T := T - 1; S[T] := ORD(S[T] ≠ S[T + 1])
                END;
           10: BEGIN T := T - 1; S[T] := ORD(S[T] < S[T + 1])
                END;
           11: BEGIN T := T - 1; S[T] := ORD(S[T] ≥ S[T + 1])
                END;
           12: BEGIN T := T - 1; S[T] := ORD(S[T] > S[T + 1])
                END;
           13: BEGIN T := T - 1; S[T] := ORD(S[T] ≤ S[T + 1])
                END;
          END;
      LOD: BEGIN T := T + 1; S[T] := S[BASE(L) + A]
          END;
      STO: BEGIN S[BASE(L) + A] := S[T]; WRITELN(S[T]); T := T - 1
          END;
      CAL: BEGIN ↱GENERATE NEW BLOCK MARK↳
             S[T + 1] := BASE(L); S[T + 2] := B; S[T + 3] := P;
             B := T + 1; P := A
          END;
      INT: T := T + A;
      JMP: P := A;
      JPC: BEGIN IF S[T] = 0 THEN P := A; T := T - 1
          END
    END ↱WITH, CASE↳
  UNTIL P = 0;
  WRITE(≡ END PL/0≡)
END ↱INTERPRET↳;
BEGIN ↱MAIN PROGRAM↳
  FOR CH := ≡A≡ TO ≡;≡ DO SSYM[CH] := NUL;
  WORD[ 1] := ≡BEGIN     ≡;    WORD[ 2] := ≡CALL      ≡;
  WORD[ 3] := ≡CONST     ≡;    WORD[ 4] := ≡DO        ≡;
  WORD[ 5] := ≡END       ≡;    WORD[ 6] := ≡IF        ≡;
  WORD[ 7] := ≡ODD       ≡;    WORD[ 8] := ≡PROCEDURE ≡;
  WORD[ 9] := ≡THEN      ≡;    WORD[10] := ≡VAR       ≡;
  WORD[11] := ≡WHILE     ≡;
  WSYM[ 1] := BEGINSYM;    WSYM[ 2] := CALLSYM;
  WSYM[ 3] := CONSTSYM;    WSYM[ 4] := DOSYM;
  WSYM[ 5] := ENDSYM;      WSYM[ 6] := IFSYM;
  WSYM[ 7] := ODDSYM;      WSYM[ 8] := PROCSYM;
  WSYM[ 9] := THENSYM;     WSYM[10] := VARSYM;
  WSYM[11] := WHILESYM;
  SSYM[≡+≡] := PLUS;       SSYM[≡-≡] := MINUS;
```

```
   SSYM[≡*≡] := TIMES;          SSYM[≡/≡] := SLASH;
   SSYM[≡(≡] := LPAREN;         SSYM[≡)≡] := RPAREN;
   SSYM[≡=≡] := EQL;            SSYM[≡,≡] := COMMA;
   SSYM[≡.≡] := PERIOD;             SSYM[≡≠≡] := NEQ;
   SSYM[≡<≡] := LSS;            SSYM[≡>≡] := GTR;
   SSYM[≡≤≡] := LEQ;            SSYM[≡≥≡] := GEQ;
   SSYM[≡;≡] := SEMICOLON;
   MNEMONIC[LIT] := ≡ LIT ≡; MNEMONIC[OPR] := ≡ OPR ≡;
   MNEMONIC[LOD] := ≡ LOD ≡; MNEMONIC[STO] := ≡ STO ≡;
   MNEMONIC[CAL] := ≡ CAL ≡; MNEMONIC[INT] := ≡ INT ≡;
   MNEMONIC[JMP] := ≡ JMP ≡; MNEMONIC[JPC] := ≡ JPC ≡;
   DECLBEGSYS := [CONSTSYM, VARSYM, PROCSYM];
   STATBEGSYS := [BEGINSYM, CALLSYM, IFSYM, WHILESYM];
   FACBEGSYS := [IDENT, NUMBER, LPAREN];
   PAGE(OUTPUT); ERR := 0;
   CC := 0; CX := 0; LL := 0; CH := ≡  ≡; KK := AL; GETSYM;
   BLOCK(0, 0, [PERIOD] + DECLBEGSYS + STATBEGSYS);
   IF SYM ≠ PERIOD THEN ERROR(9);
   IF ERR = 0 THEN INTERPRET ELSE WRITE(≡ ERRORS IN PL/0 PROGRAM≡);
99: WRITELN
END.
```